LLMBO-MO 架构蓝图

面向 Claude Code Plan Mode 的完整施工规格

> **使用说明：** 本文档描述模块架构、接口契约、数据流向和方法规格。不包含具体代码实现，供 Claude Code 在 Plan Mode 中进行逐模块讨论和实现。 **对应框架：** FrameWork.md §1–§14 + Gap Analysis G-1~G-14

---

1. 顶层目录结构

```
llambo_mo/
├── config.py              # 全局超参数、路径、变量空间定义
├── battery_model.py       # PyBaMM 仿真器（3D参数空间）
├── psi_function.py        # 物理代理函数 Ψ(θ) 及解析梯度
├── gp_model.py            # 复合GP核 + 训练 + 预测
├── tchebycheff.py         # Tchebycheff标量化 + 动态归一化
├── acquisition.py         # 采集函数 α(θ) = EI × W_charge
├── llm_interface.py       # LLM调用封装（3个触点）
├── database.py            # 实验数据库 + HV计算
├── database_summarizer.py # 无状态上下文注入器
├── optimizer.py           # 主循环 OptimizerState + Algorithm 1
├── utils.py               # 数值工具、候选验证、去重
├── visualization.py       # 图表生成（对标参考文献图1-5）
└── run.py                 # 入口点、并行调度、结果导出
```

---

2. 模块间数据流总图

```
run.py
 └──► optimizer.py (OptimizerState)
    ├──[初始化]──► llm_interface.py ──► touchpoint_1a → W矩阵
    │          └──────────────────────► touchpoint_1b → 15热启动点
    │                        └──► battery_model.py (并行×3)
    │                             └──► database.py (写入)
    │
    ├──[每轮迭代]
    │  ├──► database_summarizer.py ──► 结构化上下文字符串
    │  ├──► llm_interface.py ──────────► touchpoint_2 → 15候选点
    │  ├──► utils.py ──────────────────► 候选验证 + 去重
    │  ├──► psi_function.py ────────────► ∇Ψ(θ) for each candidate
    │  ├──► gp_model.py ────────────────► 预测 μ, σ for each candidate
    │  ├──► tchebycheff.py ──────────────► 动态归一化 + 标量化
```

```
│   ├──► acquisition.py ──────────────► α(θ) = EI × W_charge → 选3个
│   └──► battery_model.py ──────────► 并行评估3个 → database.py
│
└──[结束]──► visualization.py ──► PNG图表
    └──► run.py ──────────────► metrics_final.json
```

---

3. 各模块详细规格

3.1 `config.py` — 全局配置

职责： 集中管理所有超参数和常量，其他模块只从此处读取，不硬编码。

必须包含的配置项：

```
# 决策变量空间（3D，对应 FrameWork.md §2）
PARAM_BOUNDS = {
  "I1":  [0.01, 7.99],  # A，第一阶段电流
  "SOC1": [0.10, 0.70],  # 无量纲，切换SOC阈值
  "I2":  [0.01, 7.99]   # A，第二阶段电流
}
PARAM_NAMES = ["I1", "SOC1", "I2"]
PARAM_DIM = 3

# 目标函数（3个，均最小化）
OBJECTIVE_NAMES = ["t_charge", "T_peak", "delta_Q_aging"]

# 算法超参数（FrameWork.md §5）
N_INIT      = 15      # 初始热启动点数
N_CAND      = 15      # LLM每轮生成候选数
N_SELECT    = 3       # 每轮实际评估数
T_MAX       = 50      # 最大迭代轮数
ETA         = 0.05    # Tchebycheff增广项系数
GAMMA_INIT  = 0.1     # 初始核耦合权重（manuscript1 Eq.7起始值）
ALPHA_MAX   = 0.7     # σ上界
ALPHA_MIN   = 0.05    # σ下界
T_DECAY     = 60      # σ衰减时间常数
KAPPA       = 0.20    # EI探索-利用权衡
EPS_SIGMA   = 0.001   # σ最小值截断
RHO         = 0.1     # 停滞扩展因子

# 仿真参数
SOC_START   = 0.10
SOC_END     = 0.80
TIMEOUT_S   = 7200    # 仿真超时秒数
PENALTY_MULT = 1.20   # 超时惩罚倍率（× f_tch_max_observed）
N_PARALLEL  = 3       # 并行仿真进程数

# HV参考点（归一化空间，略大于1）
HV_REF_POINT = [1.10, 1.10, 1.10]

# LLM配置
LLM_MODEL   = "gpt-4o"  # 或 "deepseek-reasoner"
LLM_TEMP    = 0.7
```

3.2 `battery_model.py` — PyBaMM 仿真器

职责： 接收决策变量 $\theta = (I_1, SOC_1, I_2)$，返回三个目标值。支持正常评估和全量评估两种模式。

必须实现的 SEI 子模型配置（FrameWork.md §10 + 设计决策Q4）：

```
pybamm.lithium_ion.SPMe(options={
  "SEI": "reaction limited",
  "lithium plating": "irreversible",
  "lithium plating porosity change": "true"
})
参数集：Chen2020
```

类：`BatterySimulator`

方法	输入	输出	说明
`__init__()`	—	—	构建PyBaMM模型，预热
`evaluate(theta)`	`dict{I1,SOC1,I2}`	`EvalResult`	正常评估，返回3目标值
`evaluate_full(theta)`	`dict{I1,SOC1,I2}`	`EvalResult + solution`	额外缓存PyBaMM solution对象（供绘图用）
`_build_protocol(theta)`	dict	pybamm experiment	构建两阶段CC充电实验
`_extract_objectives(solution)`	solution	tuple(t,T,ΔQ)	从solution提取三目标值
`_extract_aging(solution)`	solution	float	SEI LLI + Plating LLI 之和（库仑）
`_handle_timeout(f_tch_max)`	float	EvalResult	超时惩罚：f_tch_max×1.2，is_valid=False

`EvalResult` 数据结构：

```
EvalResult:
  theta: dict        # 输入参数
  t_charge: float     # 秒
  T_peak: float       # K
  delta_Q: float      # C（库仑）
  is_valid: bool      # False=超时或仿真失败
  solution: optional   # 仅evaluate_full模式
```

关键实现注意点：

- ΔQ_aging 计算：`solution["Loss of lithium to SEI [mol]"] + solution["Loss of lithium to lithium plating [mol]"]`，乘以 Faraday 常数后转换为 Ah
- 超时时不直接赋 7200s，而调用 `_handle_timeout(f_tch_max_observed)`

- `evaluate_full` 专门用于 Pareto 前沿代表点的时序曲线绘图，不在优化主循环中调用

3.3 `psi_function.py` — 物理代理函数

职责： 实现 Ψ(θ) 及其对三个变量的解析梯度（FrameWork.md §3, Eqs.6–10）。

物理定义：

```
Ψ(θ) = I₁² · R̄₁ · t₁(SOC₁) + I₂² · R̄₂ · t₂(SOC₁)

其中：
- t₁(SOC₁) = 估计的第一阶段充电时长（由SOC₁积分估算）
- t₂(SOC₁) = 估计的第二阶段充电时长
- R̄₁, R̄₂ = 两阶段平均内阻（从Chen2020参数集查表）
```

类： `PhysicsProxy`

方法	输入	输出	说明
`__init__(R1_bar, R2_bar, capacity)`	floats	—	初始化内阻和容量参数
`compute(theta)`	`array[3]`	`float`	计算 Ψ(θ) 标量值
`gradient(theta)`	`array[3]`	`array[3]`	解析梯度 [∂Ψ/∂I₁, ∂Ψ/∂SOC₁, ∂Ψ/∂I₂]
`gradient_batch(thetas)`	`array[N,3]`	`array[N,3]`	批量梯度（供GP核矩阵构建）
`_estimate_t1(SOC1, I1)`	floats	float	由SOC差值和电流估算第一阶段时长
`_estimate_t2(SOC1, I2)`	floats	float	估算第二阶段时长

解析梯度公式（FrameWork.md Eqs.8–10）：

```
∂Ψ/∂I₁ = 2 · I₁ · R̄₁ · t₁ + I₁² · R̄₁ · (∂t₁/∂I₁)
∂Ψ/∂SOC₁ = I₁² · R̄₁ · (∂t₁/∂SOC₁) + I₂² · R̄₂ · (∂t₂/∂SOC₁)
∂Ψ/∂I₂ = 2 · I₂ · R̄₂ · t₂ + I₂² · R̄₂ · (∂t₂/∂I₂)
```

备用数值梯度（中心差分，δ=1e-5）：

- 当解析梯度数值异常时（如 SOC_1 边界附近），自动切换到数值梯度

3.4 `gp_model.py` — 复合GP核模型

职责： 实现 FrameWork.md §4 规定的复合核，训练GP，提供预测接口。

核函数定义（Eq.11）：

```
k(θ, θ') = k_RBF(θ, θ') + γ · ∇Ψ(θ)ᵀ · W^(t) · ∇Ψ(θ')

其中 W^(t) = α_t · W_time + β_t · W_temp + δ_t · W_aging（Eq.11'）
W矩阵在迭代过程中保持固定（由 touchpoint_1a 推理一次）
权重 α_t, β_t, δ_t 随时间更新（由 γ 动态更新驱动）
```

类： `CompositeGPModel`

方法	输入	输出	说明
`__init__(physics_proxy, W_matrices)`	—	—	注入物理代理和W矩阵
`fit(X, Y_scalarized, gamma)`	arrays	—	拟合GP，X为归一化参数，Y为标量化目标
`predict(X_new)`	`array[M,3]`	`(mu, sigma)`	返回均值和标准差
`compute_kernel(X1, X2, gamma)`	arrays	`array[N,M]`	构建核矩阵
`_rbf_kernel(X1, X2)`	arrays	array	标准RBF核
`_physics_kernel(X1, X2, gamma)`	arrays	array	γ · ∇Ψᵀ · W · ∇Ψ 外积核
`_validate_psd(K)`	matrix	matrix	特征值投影保证PSD（负特征值截零）
`update_gamma(gamma_new)`	float	—	更新核参数，触发重新拟合

W矩阵数据流：

```
touchpoint_1a（初始化，一次性）→ llm_interface.py → W_time, W_temp, W_aging (3×3 matrices)
→ 存入 OptimizerState.W_matrices
→ 每轮传入 CompositeGPModel.fit()
```

注意： GP 为每个目标函数单独训练一个（共3个），或训练一个标量化后的目标。根据 FrameWork.md §4，对**标量化值**训练单GP。

3.5 `tchebycheff.py` — 标量化与归一化

职责： 实现 Eq.1（增广Tchebycheff）和 Eq.2（动态归一化），严格执行硬更新。

标量化公式（Eq.1）：

```
f_tch(θ|λ) = max_i[λ_i · f̃_i(θ)] + η · Σ_i[λ_i · f̃_i(θ)]
```

对老化目标的对数变换（Eq.2a）：

```
f̃_aging = log(1 + f_aging)  （在归一化之前应用）
```

动态归一化（Eq.2b，硬更新）：

```
f̃_i = (f_i - f̃_min_i) / (f̃_max_i - f̃_min_i)
更新规则：f̃_min_i = min over ALL historical data（非指数平滑）
```

类： `TchebycheffScalarizer`

方法	输入	输出	说明
`__init__(eta, n_objectives)`	—	—	初始化
`update_bounds(F_all)`	`array[N,3]`	—	硬更新 f̃_min, f̃_max（非指数平滑）
`normalize(F_all)`	`array[N,3]`	`array[N,3]`	对所有历史数据重新归一化
`scalarize(F_normalized, lambda_vec)`	arrays	`array[N]`	对N个点计算Tchebycheff值
`scalarize_single(f_norm, lambda_vec)`	arrays	float	单点标量化
`apply_log_transform(F)`	`array[N,3]`	`array[N,3]`	仅对第三列（aging）取log

关键约束：

- `update_bounds` 每轮迭代必须传入**全量历史数据**，不能只传增量
- 归一化后所有历史数据的标量化值随之更新（GP 训练数据随之变化）
- `f̃_min` 和 `f̃_max` 不做平滑，直接取全局最小/最大值

3.6 `acquisition.py` — 采集函数

职责： 实现 FrameWork.md §5 规定的 α(θ) = EI × W_charge，以及 σ 的灵敏度引导搜索（Eqs.18–22）。

采集函数定义（Eq.14）：

$\alpha(\theta) = EI(\theta) \times W_charge(\theta)$

$EI(\theta) = (\mu^* - \mu(\theta)) \cdot \Phi(z) + \sigma(\theta) \cdot \phi(z)$
$z = (\mu^* - \mu(\theta)) / \sigma(\theta)$
μ^* = 当前最佳标量化值（动态漂移，Eqs.18–19）

$W_charge(\theta) = \exp(-\|\theta - \theta_best\|^2 / (2 \cdot \sigma_global^2))$
σ_global = per-dimension，由 Eq.20 计算，每轮一次性计算

μ 动态漂移（Eqs.18–19）：

$\mu^* = \min(f_tch_observed) \times (1 - drift_factor)$
drift_factor 随迭代递减（鼓励早期探索，后期利用）

σ 灵敏度引导（Eq.20，全局per-dimension）：

$\sigma_d = \alpha_max \cdot \exp(-t / t_decay) + \alpha_min$ for d=1,2,3
（在 θ_best 处计算，每轮调用一次，结果共享给所有候选点）

停滞扩展（Eq.22）：

若连续 $\kappa \cdot T_max$ 轮 HV 无改进，则 $\sigma_d \times (1+\rho)$

类： `AcquisitionFunction`

方法	输入	输出	说明
`__init__(kappa, eps_sigma, alpha_max, alpha_min, t_decay, rho)`	—	—	—
`compute_sigma_global(t, theta_best, gp_model)`	—	`array[3]`	Eq.20，每轮调用一次
`compute_W_charge(theta_j, theta_best, sigma_global)`	arrays	float	单候选点的空间权重
`compute_EI(mu, sigma, mu_star)`	floats	float	标准EI公式
`score(theta_j, gp_model, mu_star, sigma_global, theta_best)`	—	float	α(θ_j) = EI × W_charge
`score_batch(candidates, gp_model, mu_star, sigma_global, theta_best)`	—	`array[N]`	批量评分，选top-N_SELECT
`check_stagnation(hv_history, kappa, T_max)`	—	bool	是否触发停滞扩展
`expand_sigma(sigma_global, rho)`	array	array	停滞时扩展搜索范围

3.7 `llm_interface.py` — LLM 调用封装

职责： 封装三个 LLM 触点，所有调用无状态（上下文由 DatabaseSummarizer 注入）。

三个触点的调用时机和频率：

触点	调用时机	频率	返回值
`touchpoint_1a`	初始化阶段	仅一次	W_time, W_temp, W_aging（3个3×3矩阵）
`touchpoint_1b`	初始化阶段	仅一次	15个热启动候选点（dict列表）
`touchpoint_2`	每轮迭代开始	每轮一次	15个候选点（dict列表）

类： `LLMInterface`

方法	输入	输出	说明
`__init__(model, temperature, api_key)`	—	—	—
`infer_coupling_matrices(battery_context)`	str	`dict{W_time, W_temp, W_aging}`	touchpoint_1a
`generate_warmstart(context_str, n=15)`	str	`list[dict]`	touchpoint_1b
`generate_candidates(context_str, n=15)`	str	`list[dict]`	touchpoint_2，每轮迭代
`_call_api(system_prompt, user_prompt)`	strs	str	底层API调用，含重试逻辑
`_parse_candidates(llm_response)`	str	`list[dict]`	解析LLM输出为参数字典
`_parse_matrices(llm_response)`	str	`dict`	解析W矩阵（JSON格式）
`_validate_output(candidates)`	list	list	过滤超出参数范围的点

Prompt 构建职责分工：

- `LLMInterface` **只负责** API 调用和输出解析
- Prompt 的上下文部分由 `DatabaseSummarizer` 生成后传入
- System prompt 模板硬编码在 `LLMInterface` 中（battery domain 知识）

3.8 `database_summarizer.py` — 无状态上下文注入器

职责： 在每次 LLM 调用前，将结构化实验数据转换为 LLM 可理解的上下文字符串（FrameWork.md §7）。

类： `DatabaseSummarizer`

方法	输入	输出	说明
summarize_for_warmstart(battery_specs)	dict	str	touchpoint_1b 的上下文
summarize_for_iteration(database, optimizer_state, t)	—	str	touchpoint_2 的上下文（每轮）
_format_pareto_front(pareto_points)	list	str	格式化当前Pareto前沿为文本表格
_format_best_k(database, k=5)	—	str	列出目标值最好的k个协议
_format_trends(hv_history)	list	str	HV 趋势描述
_format_param_ranges(bounds)	dict	str	当前搜索范围描述
_format_lessons(lessons)	list[str]	str	格式化 lessons 列表（见 §3.9）

上下文结构示例（summarize_for_iteration 输出格式）：

```
[Battery System]
Cell: LG INR-21700-M50T (5Ah NMC811/Graphite)
Parameters: I1∈[0.01,7.99]A, SOC1∈[0.1,0.7], I2∈[0.01,7.99]A
Objectives (minimize): t_charge(s), T_peak(K), ΔQ_aging(C)

[Current State - Iteration 12/50]
Evaluations so far: 51
HV progress: 0.821 → 0.854 → 0.867

[Pareto Front (8 points)]
| I1   | SOC1 | I2   | t_charge | T_peak | ΔQ    |
| 3.20 | 0.35 | 2.10 | 1823     | 308.2  | 4.31  |
...

[Learned Patterns]
- High I1 (>5A) consistently raises T_peak above 315K
- SOC1 near 0.35 appears in 6/8 Pareto points

[Task]
Generate 15 new candidate protocols (I1, SOC1, I2) to evaluate next.
Return as JSON array.
```

3.9 database.py — 实验数据库

职责： 持久化所有评估结果，计算HV，维护Pareto前沿。

类： `ExperimentDatabase`

方法	输入	输出	说明
`__init__(save_path)`	str	—	初始化，支持从CSV恢复
`add(eval_result)`	`EvalResult`	—	添加评估结果
`get_all_valid()`	—	`DataFrame`	返回所有 is_valid=True 的记录
`get_all()`	—	`DataFrame`	返回全部记录（含超时点）
`get_pareto_front()`	—	`DataFrame`	计算并返回当前Pareto前沿
`compute_hv(ref_point)`	array	float	归一化空间中的HV值
`get_f_tch_max()`	—	float	当前最大Tchebycheff值（用于超时惩罚）
`to_csv(path)`	str	—	导出完整记录
`solution_cache`	—	dict	`{theta_hash: solution}` 存储 evaluate_full 结果

lessons 提取规则（解决 G-13，OptimizerState.lessons）：

lessons 由 `ExperimentDatabase` 的 `extract_lessons()` 方法生成，规则如下：

```
extract_lessons(database, window=10) → list[str]

规则1（相关性观察）：
  对每个参数维度 d，计算 corr(param_d, objective_i) over 最近 window 个点
  若 |corr| > 0.6，生成规则："High {param} correlates with high {obj}"

规则2（Pareto 集中趋势）：
  计算 Pareto 前沿点的参数均值 ± std
  生成规则："{param} in Pareto front: mean={x:.2f} ± {s:.2f}"

规则3（约束识别）：
  若某参数值域内有明显的 is_valid=False 区域（超时集中），生成警告规则

更新时机：每隔5轮迭代更新一次，追加不重复规则，最多保留20条
```

3.10 `optimizer.py` — 主循环

职责： 持有 `OptimizerState`，实现 FrameWork.md Algorithm 1 的完整5步初始化 + 37步主循环。

类： `OptimizerState`（纯数据容器）

```
OptimizerState:
  database: ExperimentDatabase
  gp_model: CompositeGPModel
  scalarizer: TchebycheffScalarizer
  acquisition: AcquisitionFunction
  physics_proxy: PhysicsProxy
  W_matrices: dict        # touchpoint_1a 推理结果，初始化后不变
  gamma: float            # 动态更新（manuscript1 Eq.7）
  sigma_global: array[3]    # 每轮更新
  theta_best: array[3]      # 当前最佳点
  hv_history: list[float]   # 每轮HV值
  pareto_size_history: list[int]
  lessons: list[str]        # extract_lessons 输出
  t: int                    # 当前迭代轮数
  f_min_prev: float         # 上一轮最小标量化值（用于γ更新）
```

类： `LLAMBOMOOptimizer`

方法	说明
`__init__(config)`	初始化所有子模块
`initialize()`	Algorithm 1 步骤1–5：调用1a推理W矩阵，调用1b生成热启动点，并行评估，训练初始GP
`run()`	执行完整优化循环（T_MAX轮）
`_iteration_step(t)`	单轮迭代逻辑（Algorithm 1 步骤6–42）
`_update_gamma(f_min_current)`	manuscript1 Eq.7：γ_{t+1} = γ_t × (1 + 0.1 × (f_min,t−f_min,t-1)/f_min,t-1)
`_retrain_gp()`	用全量归一化后数据重训练GP
`_select_top_candidates(candidates, scores)`	按α值选 N_SELECT=3 个点

单轮迭代流程（Algorithm 1 对应）：

```
_iteration_step(t):
1. database_summarizer.summarize_for_iteration() → context_str
2. llm_interface.touchpoint_2(context_str) → 15 raw candidates
3. utils.validate_and_deduplicate(candidates, database) → valid candidates
4. psi_function.gradient_batch(candidates) → ∇Ψ for all
5. gp_model.predict(candidates) → μ, σ for all
6. acquisition.compute_sigma_global(t, theta_best, gp_model)
7. acquisition.score_batch(candidates, ...) → scores
8. select top N_SELECT=3 candidates
9. battery_model.evaluate_parallel(top_3) → 3 EvalResults
10. database.add(results)
11. scalarizer.update_bounds(database.get_all_valid().objectives) ← 硬更新
12. scalarizer.normalize(all_objectives) → 重新归一化所有历史数据
13. _retrain_gp() ← 用新的归一化数据
14. _update_gamma(f_min_current)
15. state.hv_history.append(database.compute_hv(HV_REF_POINT))
16. 每5轮：database.extract_lessons() → state.lessons
17. acquisition.check_stagnation() → 必要时 expand_sigma
```

3.11 `utils.py` — 候选验证与去重

职责： 实现候选点的边界检查和去重逻辑（对应 Gap G-7，LLAMBO论文发现无此步骤接受率降至69%）。

函数：

函数	输入	输出	说明
`validate_bounds(candidates, bounds)`	list, dict	list	过滤超出参数范围的候选点
`deduplicate(candidates, database, tol=1e-3)`	list, db	list	去除与历史点距离小于tol的点
`validate_and_deduplicate(candidates, database, bounds)`	—	list	组合调用，不足N_CAND时不补充
`normalize_theta(theta, bounds)`	array, dict	array	归一化到[0,1]^3
`denormalize_theta(theta_norm, bounds)`	array, dict	array	反归一化
`theta_to_array(theta_dict)`	dict	array	dict → numpy array
`array_to_theta(arr, param_names)`	array, list	dict	numpy array → dict
`compute_pareto_front(F)`	`array[N,3]`	`array[M,3]`	非支配排序

3.12 `visualization.py` — 图表生成

职责： 生成对标参考文献图1–5的全套图表。

类： `ResultVisualizer`

方法	对应参考图	输出文件
`plot_pareto_3d(database, highlight_points)`	图1	`pareto_front_3d.png`
`plot_pareto_2d_projections(database)`	图1变体	`pareto_front_2d_projections.png`
`plot_timeseries(solution, protocol_name)`	图2/3	`protocols/{name}.png`
`plot_timeseries_overlay(solutions, names)`	图3风格	`protocols/overlay.png`
`plot_hv_convergence(hv_history, baseline=None)`	图5	`hv_convergence.png`
`plot_pareto_size_convergence(size_history)`	图4	`pareto_size_convergence.png`
`plot_gamma_evolution(gamma_history)`	调试图	`gamma_evolution.png`
`export_metrics(database, state)`	—	`metrics_final.json`
`select_representative_protocols(database, n=6)`	—	list of EvalResult

`plot_pareto_3d` 规格：

```
坐标轴：Temperature/K × Charging Time/s × Lithium-ion loss/C
散点着色：按迭代轮次用colormap（早期→晚期，蓝→红）
红星标注：用 select_representative_protocols() 返回的代表点（类比参考图A-F）
图例：colorbar表示迭代进度
```

`plot_timeseries` 规格（4子图布局，对应参考图2/3）：

```
subplot(a): Voltage/V vs Time/s，标注上限4.2V虚线
subplot(b): Temperature/K vs Time/s
subplot(c): Input Current/A vs Time/s（阶梯函数）
subplot(d): Lithium-ion loss/C vs Time/s（累积量）
```

`export_metrics` 输出格式：

```
json
```

```
{
  "final_hv": float,
  "hv_at_50_evals": float,
  "hv_at_100_evals": float,
  "hv_at_165_evals": float,
  "pareto_front_size": int,
  "best_t_charge_s": float,
  "best_T_peak_K": float,
  "best_delta_Q_C": float,
  "total_evaluations": int,
  "valid_evaluations": int,
  "timeout_count": int,
  "llm_calls_total": int,
  "gamma_final": float,
  "runtime_minutes": float
}
```

3.13 `run.py` — 入口点与并行调度

职责： 命令行入口，管理 ProcessPoolExecutor，导出结果。

命令行参数：

```
--config   配置文件路径（可选，覆盖config.py默认值）
--seed     随机种子
--output   结果输出目录（默认./results/）
--baseline 若设置，同时运行ParEGO baseline做对比
--plot-only 仅重新生成图表（不重跑优化）
```

并行仿真设计（ProcessPoolExecutor(max_workers=3)）：

```
parallel_evaluate(theta_list: list[dict]) → list[EvalResult]:
  使用 ProcessPoolExecutor(N_PARALLEL=3) 并行调用 BatterySimulator.evaluate()
  每个worker独立构建PyBaMM模型（不共享状态）
  超时通过 concurrent.futures.TimeoutError 捕获
  结果按输入顺序返回
```

输出目录结构：

```
results/
├── metrics_final.json
├── database_final.csv
├── run_log.txt
├── pareto_front_3d.png
├── pareto_front_2d_projections.png
├── hv_convergence.png
├── pareto_size_convergence.png
├── gamma_evolution.png
└── protocols/
    ├── protocol_fastest.png
    ├── protocol_coolest.png
    ├── protocol_least_aging.png
    └── protocol_knee.png
```

---

4. 关键数据契约（跨模块接口）

4.1 `theta` 的统一表示

所有模块使用 **dict** 格式作为对外接口，内部计算使用 numpy array：

```
theta_dict = {"I1": 3.2, "SOC1": 0.35, "I2": 2.1}  # 外部接口
theta_arr  = np.array([3.2, 0.35, 2.1])            # 内部计算
# 转换由 utils.theta_to_array / array_to_theta 负责
```

4.2 `EvalResult` 流转路径

```
battery_model.evaluate() → EvalResult
  → database.add()
  → tchebycheff.update_bounds() / normalize()
  → gp_model.fit()（归一化后数据）
  → database.compute_hv()
  → visualization.export_metrics()
```

4.3 GP 训练数据格式

```
X_train: array[N, 3]  # 归一化参数，utils.normalize_theta 处理
Y_train: array[N]     # Tchebycheff标量化值，scalarizer.scalarize处理
# 注意：每轮迭代归一化边界更新后，X_train 不变，Y_train 需用新边界重新计算
```

4.4 W 矩阵格式

```
W_matrices = {
   "W_time":  np.array shape(3,3),  # 物理耦合矩阵
   "W_temp":  np.array shape(3,3),
   "W_aging": np.array shape(3,3)
}
# 由 touchpoint_1a 推理，存入 OptimizerState.W_matrices
# 传入 CompositeGPModel.__init__()，整个优化过程不更新
```

---

5. 实现优先级与依赖顺序

```
阶段1（基础层，无外部依赖）:
  config.py → utils.py → psi_function.py

阶段2（仿真层）:
  battery_model.py（依赖config）

阶段3（核心算法层）:
  tchebycheff.py → gp_model.py（依赖psi_function）→ acquisition.py

阶段4（数据与接口层）:
  database.py → database_summarizer.py → llm_interface.py

阶段5（主控层）:
  optimizer.py（依赖所有上层）

阶段6（输出层）:
  visualization.py → run.py
```

---

6. Gap Analysis 对应关系

Gap ID	问题	修复模块	修复方法
G-1	决策变量4D→3D	config.py	PARAM_BOUNDS 改为3D（含SOC1=[0.1,0.7]）
G-2	psi_function缺失	psi_function.py	新建，含解析梯度
G-3	核函数错误	gp_model.py	用∇Ψ外积核替换数值相似度核

Gap ID	问题	修复模块	修复方法
G-4	采集函数错误	acquisition.py	用EI×W_charge替换PE-GenBO
G-5	SEI子模型错误	battery_model.py	reaction limited + irreversible plating
G-6	超时惩罚错误	battery_model.py	_handle_timeout用f_tch_max×1.2
G-7	超参数不匹配	config.py	对齐所有超参数
G-8	归一化平滑更新	tchebycheff.py	update_bounds用硬更新
G-9	γ更新用HV增量	optimizer.py	_update_gamma用manuscript1 Eq.7
G-10	耦合矩阵每轮推理	llm_interface.py + optimizer.py	touchpoint_1a仅初始化时调用一次
G-11	缺DatabaseSummarizer	database_summarizer.py	新建，统一上下文注入
G-12	N_select未实现	acquisition.py + optimizer.py	score_batch返回top-3
G-13	lessons算法未定义	database.py	extract_lessons()规则化实现
G-14	无并行仿真	run.py	ProcessPoolExecutor(N_PARALLEL=3)